Here's how to run the **BlinkWithoutDelay** lab on your Fedora system.

What the example does

It blinks the onboard LED (or an external LED on pin 13) **without** using delay(). This keeps the sketch responsive so other code can run at the same time. It uses millis() to track time.

Hardware

- Any Arduino board (Uno, Nano, etc.)
- Optional: external LED + 220 Ω resistor (most boards already have an onboard LED on pin 13 / LED_BUILTIN)

1. Install Arduino IDE 2 on Fedora

Recommended (Flatpak):

Bash

```
None
flatpak remote-add --if-not-exists flathub
https://flathub.org/repo/flathub.flatpakrepo
flatpak install flathub cc.arduino.IDE2
```

USB permissions (required to upload):

Bash

```
None
sudo usermod -aG dialout $USER
```

Then **log out and back in** (or reboot).

Alternatively create /etc/udev/rules.d/50-arduino.rules with:

text

```
None
KERNEL=="ttyUSB[0-9]*",MODE="0666"
KERNEL=="ttyACM[0-9]*",MODE="0666"
```

and run sudo udevadm control --reload-rules && sudo udevadm trigger.

2. Open the example

1. Launch Arduino IDE:
   Bash

```
None
   2. flatpak run cc.arduino.IDE2
```

3. Go to **File** → **Examples** → **02.Digital** → **BlinkWithoutDelay**
   (or paste the code below).

3. The sketch

C++

```
None
/*
  Blink without Delay
  https://docs.arduino.cc/built-in-examples/digital/BlinkWithoutDelay/
*/

const int ledPin = LED_BUILTIN;  // onboard LED

int ledState = LOW;
unsigned long previousMillis = 0;
const long interval = 1000;      // blink every 1000 ms (1 second)

void setup() {
  pinMode(ledPin, OUTPUT);
}

void loop() {
  unsigned long currentMillis = millis();

  if (currentMillis - previousMillis >= interval) {
    previousMillis = currentMillis;

    // toggle LED
    ledState = (ledState == LOW) ? HIGH : LOW;
    digitalWrite(ledPin, ledState);
  }

  // other non-blocking code can go here
```

```
}
```

4. Upload and run

1. Connect the Arduino via USB.
2. In the IDE: **Tools** → **Board** → select your board.
3. **Tools** → **Port** → select the correct port (usually /dev/ttyACM0 or /dev/ttyUSB0).
4. Click **Upload**.
5. The onboard LED should blink every second **without** blocking the program.

Notes

- Change interval to make it blink faster/slower (e.g. 500 for 0.5 s).
- Because it doesn't use delay(), you can add other code inside loop() that still runs while the LED is blinking.
- If the port doesn't appear, double-check the dialout group membership and udev rules, then reconnect the board.

Would you like help installing the IDE, selecting a specific board, or modifying the sketch (e.g. different blink timing or multiple LEDs)?